

# Manual de usuario y Guía técnica

Almuerzos Inclusivos – Talleres Esperanza

## Guía de navegación del manual

- Sección A Manual para usuarios finales
- Sección B Guía técnica para desarrolladores y mantenedores

## Sección A Manual para usuarios finales

### A.1 Propósito y alcance

Este manual explica cómo utilizar la aplicación web para gestionar pedidos de almuerzos en Talleres Esperanza. El enfoque es inclusivo, con lenguaje claro y pasos guiados para que cualquier persona de la comunidad pueda realizar pedidos con mínima asistencia.

### A.2 Requisitos para el uso diario

- Navegador recomendado Google Chrome o equivalente actualizado.
- Conexión a internet Preferible. En caso de no contar con internet, el pedido puede prepararse y exportarse como PDF o Excel para enviarlo por otro canal.
- Dispositivo Equipo de escritorio o portátil con teclado y mouse. La interfaz también funciona en tablet o celular.

### A.3 Roles y responsabilidades operativas

Aunque el sistema actual no usa autenticación, se recomienda definir responsabilidades claras para garantizar orden y trazabilidad.

- Captura de pedidos: Persona encargada de registrar las selecciones de los aprendices y docentes.
- Validación: Responsable de revisar el resumen antes de exportar.
- Envío del consolidado: Persona que envía el PDF o Excel al restaurante y guarda una copia en el archivo de la fundación.
- Archivo y control: Persona que organiza los archivos por fecha y atiende solicitudes de consulta.

### A.4 Descripción detallada de vistas y procedimientos

#### A.4.1 Inicio

Objetivo Presentar un punto de partida simple con dos accesos Hacer pedido y Ver historial. El texto es de gran tamaño y los botones son fácilmente identificables.

- Abrir la aplicación en el navegador.
- Leer el texto de bienvenida y seleccionar Hacer pedido para iniciar el flujo.

#### **A.4.2 Hacer pedido**

Objetivo: Mostrar la lista de personas organizadas por categoría para facilitar la selección. El listado incluye estudiantes y profesores.

- Desplazarse por el listado hasta ubicar a la persona. Los nombres están agrupados por categoría.
- Seleccionar a la persona para abrir la configuración de su pedido.

Validaciones: El sistema puede recordar la última persona atendida durante la sesión para acelerar pedidos consecutivos.

Recomendaciones: Usar un orden predefinido por lista y verificar que ninguna persona quede sin seleccionar si corresponde.

#### **A.4.3 Opciones del pedido**

Objetivo: Configurar el pedido de la persona seleccionada en pasos sencillos. Las opciones se muestran como elecciones claras de alimentos y bebidas.

- Elegir una opción entre fruta o sopa.
- Elegir una opción entre jugo o limonada.
- Seleccionar el plato principal.
- Agregar observaciones si se requiere por ejemplo sin sal, alergias, tamaño de porción.
- Seleccionar el método de pago efectivo o bono.
- Guardar para volver al listado y continuar con la siguiente persona.

Buenas prácticas: Confirmar verbalmente con la persona su elección antes de guardar.

#### **A.4.4 Resumen del pedido**

Objetivo: Revisar y confirmar el pedido consolidado antes de exportar. Se muestran totales y un listado por persona para detectar inconsistencias.

- Revisar totales cantidad de almuerzos, distribución por método de pago y observaciones.
- Si hay un error, regresar a la persona correspondiente, corregir y guardar.
- Exportar Consolidado en formato PDF para enviar al restaurante o en Excel para control interno.

Sugerencia: Nombrar los archivos con el formato AAAA-MM-DD Pedido Talleres Esperanza para un archivo ordenado.

#### **A.4.5 Historial**

Objetivo: Consultar pedidos de fechas anteriores, aplicar filtros por rango y descargar consolidaciones en Excel.

1. Seleccionar fecha de inicio y fin para filtrar los registros.
2. Revisar la lista de pedidos filtrados y los agregados diarios.
3. Descargar un archivo Excel si se requiere reporte administrativo.

Nota: El historial completo requiere conexión con la base de datos. En modo de datos de muestra puede estar limitado.

### **A.5 Buenas prácticas operativas diarias**

- Definir una hora de corte de pedidos para enviar al restaurante.
- Usar la pantalla Resumen para detectar pedidos duplicados o incompletos.
- Guardar el PDF y el Excel del día en una carpeta mensual, nombrados por fecha.
- Mantener confidencialidad, no publicar nombres ni fotos en canales no autorizados.
- Realizar una verificación rápida de accesibilidad tamaño de fuente legible, contraste suficiente, foco visible.

### **A.6 Solución de problemas frecuentes**

- No veo la lista de personas: Verificar conexión a internet y configuración de la base de datos. Con base de datos inactiva, la aplicación puede usar datos de muestra.
- No descarga el PDF: Revisar permisos del navegador o intentar generar Excel.
- Necesito cambiar un pedido: Regresar a la pantalla de la persona, editar y guardar. Volver a exportar el consolidado.
- Se cerró el navegador sin exportar: Volver a ingresar y revisar si la información temporal de la sesión está disponible. De lo contrario, rehacer el pedido con apoyo del listado.

## **Sección B Guía técnica para desarrolladores y mantenedores**

### **B.1 Descripción general de la arquitectura**

La aplicación es una SPA desarrollada con React y TypeScript, empacada con Vite y estilizada con Tailwind. Opera del lado del cliente y se integra opcionalmente con Supabase para persistencia en PostgreSQL. Si no hay Supabase, se apoya en datos de muestra y localStorage para la sesión.

### **B.2 Estructura del proyecto**

Archivos principales en la carpeta src.

```
src/App.tsx
src/index.css
src/main.tsx
src/vite-env.d.ts
src/components/Layout/Footer.tsx
src/components/Layout/Header.tsx
src/components/Layout/Layout.tsx
src/config/supabase.ts
src/pages/History.tsx
```

```
src/pages/Home.tsx
src/pages/MakeOrder.tsx
src/pages/OrderOptions.tsx
src/pages/OrderSummary.tsx
src/types/index.ts
```

### B.3 Instalación, ejecución y scripts

- Entrar a la carpeta project.

```
cd project
```

- Instalar dependencias.

```
npm install
```

- Ejecución en modo desarrollo.

```
vite
```

- Construcción para producción.

```
vite build
```

- Previsualización de la build.

```
vite preview
```

Linter opcional.

```
eslint .
```

### B.4 Variables de entorno y configuración

Se definen en un archivo .env en la carpeta project. Al usar Vite, los nombres deben iniciar con VITE para ser accesibles en el cliente.

```
VITE_SUPABASE_URL=https://SU_PROYECTO.supabase.co
VITE_SUPABASE_ANON_KEY=su_clave_anonima
```

Referencia de creación del cliente Supabase en código.

```
import { createClient } from '@supabase/supabase-js';
```

```
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL || 'https://placeholder.supabase.co';
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY || 'placeholder-key';
```

```
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

### B.5 Rutas, páginas y layout

Las rutas están definidas en App.tsx. Estructura general.

```

import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Layout from './components/Layout/Layout';
import Home from './pages/Home';
import MakeOrder from './pages/MakeOrder';
import OrderOptions from './pages/OrderOptions';
import OrderSummary from './pages/OrderSummary';
import History from './pages/History';

function App() {
  return (
    <Router>
      <Layout>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/make-order" element={<MakeOrder />} />
          <Route path="/order-options/:personId" element={<OrderOptions />} />
          <Route path="/order-summary" element={<OrderSummary />} />
          <Route path="/history" element={<History />} />
        </Routes>
      </Layout>
    </Router>
  );
}

```

```
export default App;
```

Páginas clave y propósito.

- Home Presenta accesos a Hacer pedido y Ver historial y un texto de orientación.
- MakeOrder Lista personas y permite navegar a la configuración de cada pedido.
- OrderOptions Configura bebida, acompañamiento, plato principal, observaciones y método de pago.
- OrderSummary Muestra totales y listado por persona. Exporta PDF y Excel y confirma el pedido.
- History Filtra por fechas, resume pedidos y descarga Excel.

## B.6 Tipos de datos en TypeScript

Interfaces que tipan las entidades del sistema para reducir errores en tiempo de compilación y mejorar la legibilidad.

```

export interface Person {
  id: string;
  name: string;
  photo: string;
}

```

```

category: 'student' | 'teacher';
created_at: string;
}

```

```

export interface Order {
  id: string;
  person_id: string;
  person_name: string;
  person_photo: string;
  fruit_or_soup: 'fruit' | 'soup' | null;
  juice_or_lemonade: 'juice' | 'lemonade' | null;
  main_dish: 'spaghetti' | 'beef' | 'chicken' | null;
  observations: string;
  payment_method: 'cash' | 'voucher';
  order_date: string;
  created_at: string;
}

```

```

export interface DailyReport {
  id: string;
  report_date: string;
  total_orders: number;
  cash_orders: number;
  voucher_orders: number;
  created_at: string;
}

```

## B.7 Esquema de base de datos en Supabase

Tablas people, orders y daily\_reports. A continuación un esquema sugerido con restricciones e índices.

```

create table if not exists people (
  id text primary key,
  name text not null,
  photo text,
  category text not null check (category in ('student','teacher')),
  created_at timestamptz default now()
);

```

```

create table if not exists orders (
  id uuid primary key default gen_random_uuid(),
  person_id text not null references people(id),
  person_name text not null,
  person_photo text,
  fruit_or_soup text check (fruit_or_soup in ('fruit','soup')),
  juice_or_lemonade text check (juice_or_lemonade in ('juice','lemonade')),

```

```

main_dish text check (main_dish in ('spaghetti','beef','chicken')),
observations text default "",
payment_method text check (payment_method in ('cash','voucher')),
order_date date not null,
created_at timestampz default now()
);

```

```

create index if not exists idx_orders_order_date on orders(order_date);
create index if not exists idx_orders_created_at on orders(created_at);

```

```

create table if not exists daily_reports (
  id uuid primary key default gen_random_uuid(),
  report_date date not null,
  total_orders int not null,
  cash_orders int not null,
  voucher_orders int not null,
  created_at timestampz default now()
);

```

```

create index if not exists idx_reports_date on daily_reports(report_date);

```

## B.8 Políticas de seguridad y RLS

En prototipo se habilita RLS con políticas permisivas para lectura y ciertas inserciones. En producción se recomienda autenticación con roles y políticas estrictas para cada tabla.

```

alter table people enable row level security;
alter table orders enable row level security;
alter table daily_reports enable row level security;

```

```

create policy anon_select_people on people for select using (true);
create policy anon_select_orders on orders for select using (true);
create policy anon_insert_orders on orders for insert with check (true);
create policy anon_select_reports on daily_reports for select using (true);
create policy anon_insert_reports on daily_reports for insert with check (true);

```

Recomendación para producción. Usar autenticación y separar permisos por rol con reglas que limiten inserciones al contexto de la institución.

```

-- Ejemplo orientativo, ajustar a su modelo de autenticación
-- Requiere usuarios autenticados y claims de rol
create policy role_reader_people on people for select using (auth.role() = 'reader');
create policy role_writer_orders on orders for insert with check (auth.role() = 'writer');
create policy role_reader_orders on orders for select using (auth.role() in ('reader','writer'));

```

## **B.9 Calidad, pruebas y criterios de aceptación**

- Usabilidad: Tareas clave completadas en menos de 90 segundos con al menos 85 por ciento de éxito en prueba guiada.
- Accesibilidad: Cumplimiento de WCAG 2.2 esencial contraste AA, foco visible, navegación por teclado, etiquetas ARIA.
- Confiabilidad: Cero fallos en build y exportación PDF y Excel inferior al 1 por ciento de fallos.
- Datos: Guardar solo los campos estrictamente necesarios y validar consentimiento para uso de nombres y fotos.

Plan de prueba mínimo.

- Prueba de flujo completo desde inicio hasta exportación PDF.
- Prueba de edición y corrección de un pedido antes de exportar.
- Prueba de filtros en Historial y descarga de Excel.
- Prueba de navegación por teclado y foco visible en los elementos interactivos.

## **B.10 Despliegue**

- Generar la build de producción.

vite build

- Subir la carpeta dist a un hosting estático Netlify, Vercel o equivalente.
- Configurar variables de entorno en el panel de despliegue.
- Configurar redirecciones para SPA Todas las rutas deben apuntar a index.html.

Verificación posterior al despliegue cargar la aplicación, realizar un pedido de prueba y exportar PDF y Excel.

## **B.11 Mantenimiento y operación**

- Mensual: Actualizar dependencias tras probar en entorno local. Ejecutar pruebas de exportación.
- Trimestral: Revisar políticas de acceso, respaldos y vigencia de claves.
- Semestral: Revisar criterios de accesibilidad y realizar encuesta de satisfacción a usuarios.
- Ante incidentes: Registrar el problema, restaurar si es necesario y documentar la causa para prevención futura.

## **B.12 Flujo de trabajo y versionado**

- Ramas por característica y pull requests pequeños con revisión entre pares.
- Mensajes de commit claros e historia limpia.
- Checklist de revisión accesibilidad, pruebas de exportación, verificación de dependencias.



## **Buenas prácticas ampliadas**

- **Accesibilidad** Diseñar y revisar con la lista esencial WCAG 2.2. Usar roles ARIA semánticos cuando corresponda y asegurar foco visible.
- **Privacidad** Usar minimización de datos y establecer tiempos de retención. Evitar almacenar información sensible innecesaria.
- **Usabilidad** Evitar textos largos en pantallas críticas. Preferir etiquetas claras y acciones visibles. Prevenir duplicidad de pedidos con confirmación.
- **Seguridad** No exponer claves en el repositorio. Rotarlas en cambios de equipo. Limitar permisos de la clave anónima.
- **Rendimiento** Revisar tamaño de los recursos estáticos e íconos. Aprovechar caché del navegador.
- **Escalabilidad** Preparar el proyecto para editor de menú, roles, modo offline y panel analítico.
- **Documentación** Mantener README, .env.template, guía técnica y manual de usuario alineados y actualizados.

## **Recomendaciones a futuro**

- **Autenticación y roles** Separar captura, supervisión y administración con permisos diferenciados.
- **Editor de menú** Gestión dinámica de platos desde la interfaz para no depender de cambios de código.
- **Apoyos aumentados** Pictogramas y lectura en voz en pantallas clave. Considerar atajos por teclado.
- **Modo offline** Ligero con sincronización cuando regrese la conectividad.
- **Panel de análisis** Métricas de demanda por día, persona y tipo de plato para planear compras y reducir desperdicios.
- **Internacionalización** Idioma alterno y temas de alto contraste conmutables.
- **QA automatizado** Integrar pruebas de accesibilidad y smoke tests en un flujo de integración continua.
- **Endurecimiento RLS** Políticas por rol y restricciones de inserción y lectura según contexto institucional.